

Controller-Themed BOP-IT

Fangchun Li, Adam Hartman, Kirk McCann

Brief Overview:

Our BOP-IT design will be themed around a gaming controller and its typical inputs. Our completed design will have a form factor similar to a Nintendo Switch/DS. The inputs we will be using are as follows:

Flick It! - Analog Stick

When a user has to Flick It!, they will have to flick the analog stick so its position is moved from its origin.

Push It! - Button

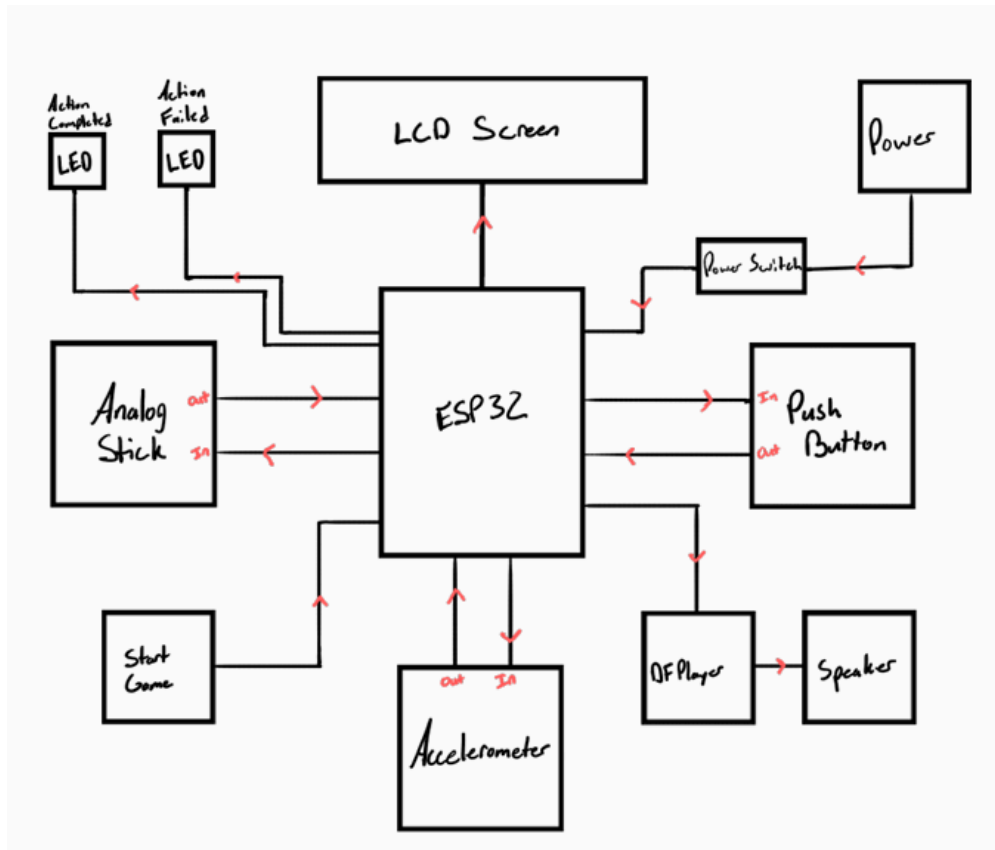
When a user has to Push It!, they will have to press a push button.

Shake It! - Accelerometer

When a user has to Shake It!, they will have to shake, or tilt the controller to get the accelerometer to register movement.

The BOP-IT will include multiple visual cues to tell the player their score, as well as successful and failed inputs. The Score is going to be displayed on an LCD Screen, while the input success or failure will be shown with LEDs. To indicate the next input while playing the game, we will use a DFPlayer connected to a speaker to audibly indicate the next action. As for power, we will use either a 9V or Lithium Ion battery that will be connected to a 5V regulator and a main power switch on the back of the controller. The entire game will be controlled using an ESP32 Microcontroller. This controller will be seated on a custom PCB that will make all the necessary connections.

Block Diagram:



Pseudo Code:

```
# Constants & Globals
SET MAX_TIME      TO 5
SET NUM_ACTIONS   TO 3
SET LED_DURATION
SET MOVEMENT_THRESHOLD
```

```
setup()
```

```
{
  SET score TO 0
  SET time  TO MAX_TIME
  SET gameOver TO true
  initializePins()
  initializeDisplay()
  initializeSound()
  SET greenLED TO LOW
  SET redLED TO LOW
}
```

```
loop()
```

```
{
  WHILE true
```

```

waitForNewGame()
SET score TO 0
SET time TO MAX_TIME
SET greenLED TO LOW
SET redLED TO LOW
update_display_Score(score)

SET gameOver TO false
WHILE gameOver is false
    SET currentAction TO generateRandom(1, NUM_ACTIONS)

    IF currentAction == 1 THEN
        playSound(ACCELEROMETER_PROMPT)
        SET result TO checkAccelerometer()
    ELSE IF currentAction == 2 THEN
        playSound(BUTTON_PROMPT)
        SET result TO checkPushButton()
    ELSE IF currentAction == 3 THEN
        playSound(JOYSTICK_PROMPT)
        SET result TO checkJoystick()
    END IF

    IF result == CORRECT THEN
        INCREMENT score BY 1
        lightLED(GREEN)
        IF score == 99 THEN
            playSound(WIN)
            update_display_Score(score)
            SET gameOver TO true
        ELSE
            playSound(SUCCESS)
            update_display_Score(score)
        END IF
    ELSE IF result == TIMEOUT OR result == INCORRECT THEN
        lightLED(RED)
        playSound(LOSE)
        update_display_Score(score)
        SET gameOver TO true
    END IF
END WHILE
END WHILE
}

```

```

waitForNewGame()
{
    SET greenLED TO LOW
    SET redLED TO LOW
    WHILE true
        READ newGameButton state
        IF newGameButton is pressed THEN
            RETURN
        END IF
    END WHILE
}

```

```

checkAccelerometer()
{
    SET time TO MAX_TIME
    WHILE time > 0
        decreaseTime()
        READ accelerometer magnitude
        READ button state
        READ joystick direction

        IF accelerometer magnitude > MOVEMENT_THRESHOLD THEN
            RETURN CORRECT
        ELSE IF button is pressed OR joystick moved THEN
            RETURN INCORRECT
        END IF
    END WHILE
    RETURN INCORRECT
}

```

```

checkPushButton()
{
    SET time TO MAX_TIME
    WHILE time > 0
        decreaseTime()
        READ button state
        READ accelerometer magnitude
        READ joystick direction

        IF button is pressed THEN
            RETURN CORRECT
        ELSE IF accelerometer magnitude > MOVEMENT_THRESHOLD OR joystick moved THEN
            RETURN INCORRECT
        END IF
    END WHILE
    RETURN INCORRECT
}

```

```

checkJoystick()
{
    SET time TO MAX_TIME
    WHILE time > 0
        decreaseTime()
        READ joystick direction
        READ button state
        READ accelerometer magnitude

        IF joystick moved in ANY direction THEN
            RETURN CORRECT
        ELSE IF button is pressed OR accelerometer magnitude > MOVEMENT_THRESHOLD THEN
            RETURN INCORRECT
        END IF
    END WHILE
    RETURN INCORRECT
}

```

```

decreaseTime()
{
    SET time TO time - time_decrement #haven't decided timing decrement yet
}

lightLED(color)
{
    IF color == GREEN THEN
        SET greenLED TO HIGH
        WAIT LED_DURATION
        SET greenLED TO LOW
    ELSE IF color == RED THEN
        SET redLED TO HIGH
    END IF
}

update_display_Score(score)
{
    DISPLAY score on screen
}

playSound(type)
{
    IF type == SUCCESS THEN play success audio
    IF type == WIN THEN play win audio
    IF type == LOSE THEN play lose audio
    IF type == ACCELEROMETER_PROMPT THEN play accelerometer prompt audio
    IF type == BUTTON_PROMPT THEN play button prompt audio
    IF type == JOYSTICK_PROMPT THEN play joystick prompt audio
}

initializePins()
{
    SET pins to i/o and initial states
}

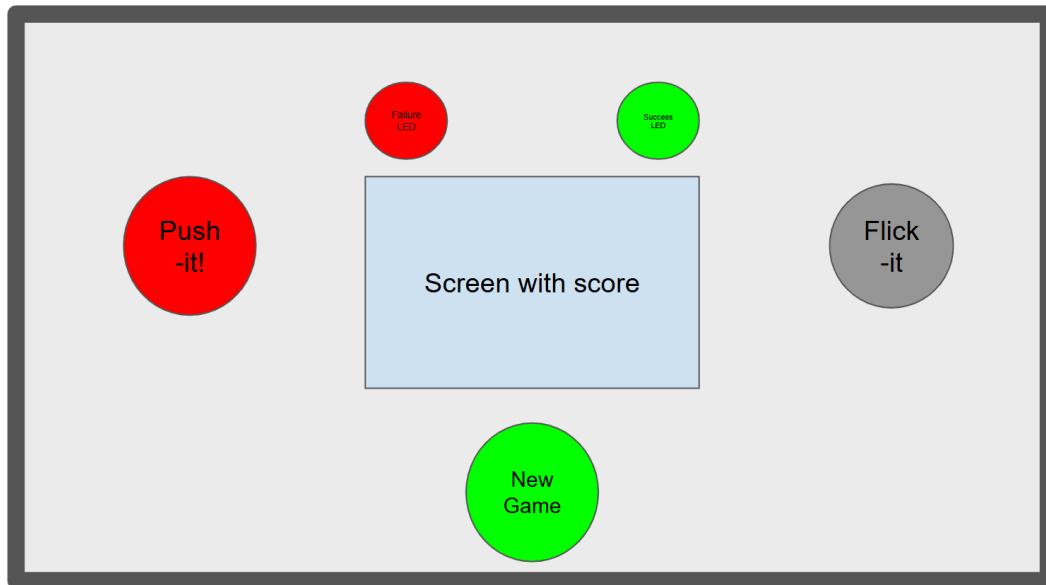
initializeDisplay()
{
    clear screen and set it up
}

initializeSound()
{
    initialize speaker and load or map sound types
}

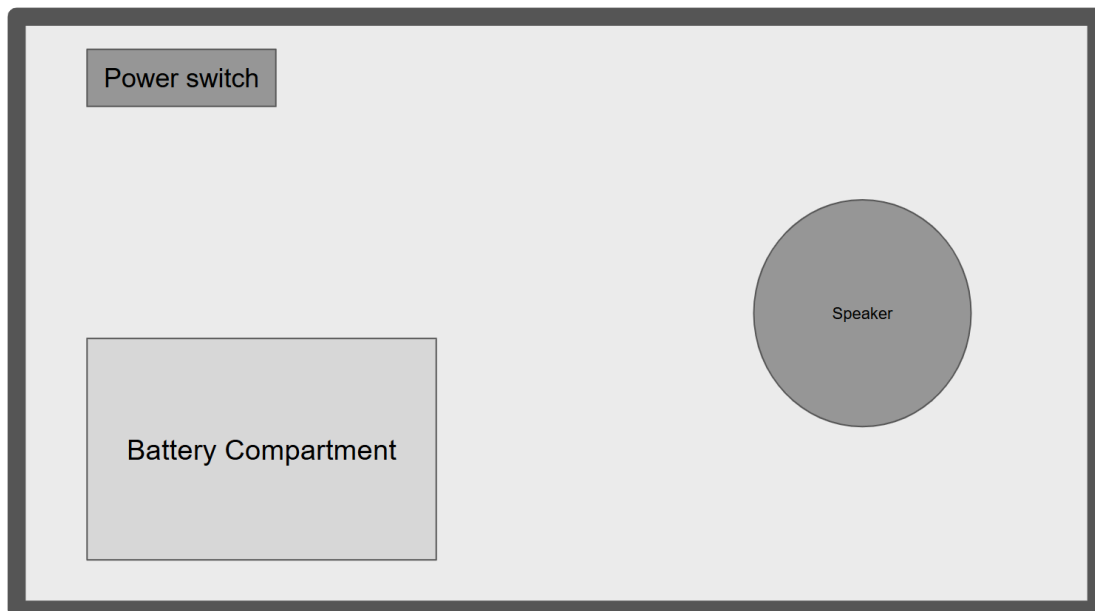
```

Enclosure Design:

Front Design



Back Design



Team Roles:

Adam: PCB and Circuit Design lead

Fangchun: Enclosure Design and hardware assembly lead
Kirk: Microcontroller Programming lead

Everyone:

- Component procurement & BOM management
- System validation & testing
- Documentation & report writing
- Cross-role collaboration and integration support

B.O.M.

Quantity	Link	Description	Pitt User ID	Unit Price	Extended Price
1	https://www.amazon.com/HiLetgo-Arduino-P	DFPlayer Audio Voice Music Module	awh26	7.99	7.99